Alumni–Student Mentorship Platform: DBMS Project Blueprint

My recommendation is to develop this as a production-style Alumni–Student Mentorship and Mock Interview Platform, with database engineering as its main strength.

The goal should be bigger than building a website where students find alumni and book interviews. Your application should demonstrate that you understand:

- How to design a relational database from business requirements.
- How to enforce data integrity through constraints and transactions.
- How to write complex and efficient SQL queries.
- How to manage concurrent users and prevent inconsistent updates.
- How to use key-value databases for low-latency access.
- How to represent connected data in a graph database.
- How to combine multiple database technologies without creating inconsistent data.

I reviewed your four-unit DBMS course outline and the existing Alumni Mentor Allocation for College Students mini-project.

The central architectural decision: Use a modular monolith with polyglot persistence, rather than implementing every component as an independent microservice.

That means your application has clearly separated business modules, but they share one primary relational database. Redis and Neo4j serve specialised workloads alongside it.

This will let you demonstrate advanced DBMS concepts without making the project unnecessarily difficult to build and maintain.

1. Project vision and scope

AlumniConnect

Proposed project identity · Alumni mentorship, career guidance and mock interview management

The problem being solved

Students often struggle to find alumni who have relevant technical skills, industry experience, and the willingness to guide them. Alumni also need a structured way to manage mentorship requests, sessions and student progress.

The proposed solution

A platform where students discover suitable alumni, request mentorship, book mock interviews, attend offline sessions, receive structured feedback and track their progress. Alumni can manage availability, approve requests and record session outcomes.

The main user roles

User	Main responsibilities
Student / Mentee	Maintain profile, find mentors, request mentorship, book sessions, submit reviews
Alumni / Mentor	Maintain professional profile, manage skills and availability, accept requests, record sessions
Administrator	Manage users, departments, skills, mentorship domains and platform analytics

I recommend including an administrator role. It gives you a meaningful use case for access control, views, aggregate queries and database reporting.

Core application modules

Module	Responsibility
Identity and Access	Registration, login, roles and permissions
Profile Management	Student and alumni professional profiles

Skill and Domain Management	Skills, experience, interests and career domains
Mentor Discovery	Search, matching and recommendations
Mentorship Requests	Request, acceptance, rejection and allocation
Availability Management	Mentor availability and preferred timings
Booking Management	Mock interview and mentorship booking
Session Management	Actual session status and outcomes
Feedback and Reviews	Structured feedback and mentor ratings
Notification Management	Booking, request and session notifications
Administration and Analytics	User management, reporting and platform insights

2. Recommended application architecture

The architecture should retain the logical responsibilities from your original design while avoiding unnecessary distributed services.

2.1 Architecture style: modular monolith + multiple databases

I would change one important aspect of your original architecture.

You listed Auth, Profile, Matching, Booking, Feedback and Notification as separate services. Their responsibilities are good, but they do not need to be independently deployed microservices.

For this project, use:

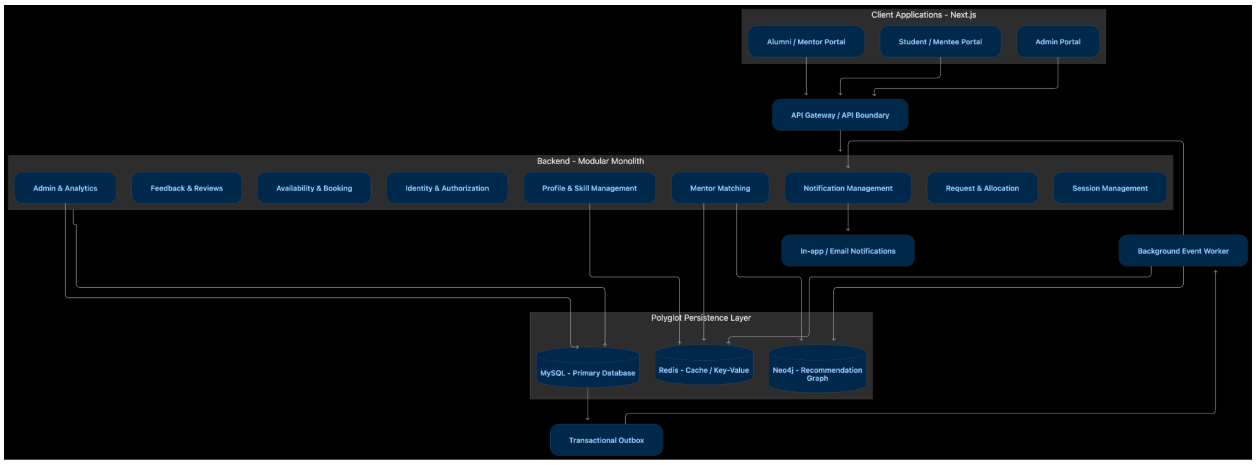
One backend application, organised into independent business modules, connected to specialised database systems.

Why?

1. Booking, allocation and session management frequently need coordinated SQL transactions.
2. A separate database for every service would introduce distributed consistency problems.
3. You are building one integrated application, not operating a large engineering organisation.
4. You still get to demonstrate multiple databases and well-defined service boundaries.

Your API Gateway remains the single entry point logically. It can be implemented as the API boundary of your backend rather than a separate deployment.

2.2 Proposed high-level architecture



The important architectural concept is that each database has a specific responsibility.

You should not simply store every record in all three databases.

Technology choices

Layer	Recommended technology	Purpose
Frontend	Next.js + TypeScript	Role-based interfaces
Backend	FastAPI	Central application API and business modules
Primary database	MySQL 8.x + InnoDB	Relational data and transactions
Cache / key-value	Redis	Fast reads and temporary state
Graph database	Neo4j	Relationship-based recommendations
Optional vector search	Redis Stack	Semantic mentor discovery
DB administration	MySQL Workbench	SQL development and inspection
Graph administration	Neo4j Browser	Graph modelling and Cypher
Cache inspection	Redis CLI	Redis CRUD and performance testing

I recommend FastAPI because it allows you to organise the different backend responsibilities cleanly, while Next.js handles the application interface. MySQL remains central to your DBMS work.

3. Database architecture: what each database should actually do

This is the most important part of the design.

Your application should demonstrate polyglot persistence: choosing different database technologies for different data access patterns.

Kanbel Technologies

1. MySQL — Primary relational database

System of record

MySQL stores the authoritative application data.

What belongs here?

- User identities, roles and permissions.
- Student and mentor profiles.
- Skills, companies, departments and domains.
- Mentorship requests and allocations.
- Availability and bookings.
- Mentorship sessions.
- Feedback and reviews.
- Notifications and audit records.

Why MySQL?

Your application has many relationships, foreign-key requirements, business rules and operations that must happen atomically.

MySQL is the right place for enforcing those rules.

It also provides a strong foundation for demonstrating your Units 1, 2, 3 and 4.

What are the pricing options for Redis?

2. Redis — Key-value database

Performance layer

Redis should make frequently accessed data faster to retrieve.

Proposed use cases:

- Cache frequently viewed mentor profiles.
- Cache popular skill and domain information.
- Cache previously calculated recommendation results.
- Store temporary login challenges and short-lived session information.
- Apply API rate limiting.
- Maintain short-lived idempotency records for requests.

Important: Do not use Redis as the authority for booking availability. MySQL must make the final booking decision.

Redis will also give you the opportunity to demonstrate strings, hashes, lists, sets, sorted sets, expiry and TTL operations taught in your Unit 4 material.

DBMS Unit 4(2).pdf
Neo4j Logo Vector Download | Logowik

3. Neo4j — Graph database

Recommendation engine

Neo4j is where the interesting recommendation work happens.

Rather than finding mentors only through large SQL joins, represent students, alumni, skills, domains and companies as connected nodes.

Then use graph traversal to discover mentors through relevant paths.

Proposed use cases:

- Skill-based mentor discovery.
- Domain-based recommendations.
- Finding alumni with relevant industry experience.
- Discovering indirect career and expertise connections.
- Explaining why a mentor was recommended.

This is a genuine use of the graph model, not simply duplicating SQL tables in Neo4j.

DBMS Unit 4(2).pdf

4. Vector search — Optional advanced extension

Phase 2

I would add semantic search after the SQL and graph recommendation systems work.

For example, a student writes:

"I want to work in machine learning infrastructure and learn how companies deploy models in production."

A traditional exact-match query might miss alumni whose profiles describe this expertise using different words.

Vector search can identify semantically similar descriptions by comparing embeddings.

My recommendation: Explore Redis Stack vector search rather than immediately introducing a fourth independent database. Your Unit 4 slides specifically cover vector search in Redis Stack, including vector indexing and similarity metrics.

DBMS Unit 4(2).pdf

The vector result should then be combined with graph relationships and SQL eligibility checks.

3.1 The source-of-truth principle

You need to establish this before designing anything else.

Information	Authoritative database
Student identity	MySQL
Mentor identity	MySQL
Student skills	MySQL
Mentor skills	MySQL
Bookings and availability	MySQL
Session status	MySQL

Ratings and feedback	MySQL
Cached mentor cards	Redis
Cached recommendations	Redis
Student–skill connections	Neo4j projection of SQL data
Mentor–skill connections	Neo4j projection of SQL data
Graph recommendation paths	Neo4j
Text embeddings	Redis Stack, optional

The distinction is important:

- MySQL owns the actual business data.
- Redis stores temporary or rebuildable data for speed.
- Neo4j stores a graph representation optimised for relationship-based queries.
- Vector indexes store searchable representations of text.

When an alumni member changes their skills in MySQL, the change must eventually reach Neo4j and the relevant Redis caches.

MySQL must always win when there is a disagreement.

4. Designing the application domain

Before creating a database schema, we should understand the actual entities, their responsibilities and how they relate.

I reviewed your existing mini-project document.

Your existing project already has 12 core entities/tables:

1. Department
2. Company
3. Skill
4. Mentorship Domain
5. Student
6. Alumni
7. Student Skill
8. Alumni Skill
9. Mentorship Request
10. Mentor Allocation
11. Mentorship Session
12. Feedback

Your existing ER and schema diagrams appear on pages 1–2, with table creation and relationship details on pages 3–8. That is a good starting foundation.

DBMS MINIPROJECT.pdf

I would retain its business domain but expand the application model.

4.1 Proposed conceptual entities

This is a domain-level design, not the physical SQL schema.

A. Identity and profile domain

- User
- Role
- Student Profile
- Mentor / Alumni Profile
- Department
- Company

Purpose: Establish one identity for each account, with different role-specific profile information.

B. Skills and expertise domain

- Skill
- Student Skill
- Mentor Skill
- Mentorship Domain
- Student Domain Interest
- Mentor Domain Expertise

Purpose: Represent many-to-many relationships between students, mentors, skills and domains.

C. Matching and allocation domain

- Mentorship Request
- Recommendation Run
- Recommended Mentor
- Mentor Allocation

Purpose: Separate potential mentor recommendations from actual accepted mentorship relationships.

D. Scheduling and session domain

- Mentor Availability
- Booking
- Mentorship Session

Purpose: Handle scheduling, reservations and actual offline meeting outcomes independently.

E. Feedback and communication domain

- Session Feedback
- Mentor Review
- Notification
- Notification Outbox
- Audit Log

Purpose: Capture interaction quality, ratings, notifications and important historical changes.

4.2 Important relationships

Here is how the main entities should behave:

Relationship	Cardinality	Meaning
User → Student Profile	1 : 0..1	An account may have a student profile
User → Mentor Profile	1 : 0..1	An account may have a mentor profile

Student ↔ Skill	M : N	A student can have multiple skills
Mentor ↔ Skill	M : N	A mentor can have multiple skills
Student → Mentorship Request	1 : N	A student can send multiple requests
Request → Allocation	1 : 0..1	A request can result in at most one allocation
Allocation → Session	1 : N	One mentorship relationship can have multiple sessions
Mentor → Availability	1 : N	A mentor can publish multiple availability slots
Availability → Booking	1 : 0..1	A specific slot can be booked at most once
Booking → Session	1 : 0..1	A confirmed booking can produce a session record

There are two design decisions here that I consider particularly important.

Decision 1: Separate User from Student and Mentor profiles

Your current design has different Student and Alumni entities.

I would introduce a common account identity and keep their student/mentor information in separate profile concepts.

Why?

An account should have one authentication identity. Its role-specific information should not need to duplicate login credentials and common identity information.

You can still retain separate Student and Alumni profile concepts.

Decision 2: Separate Booking from Mentorship Session

This distinction is essential for your application.

A booking represents a planned appointment.

A mentorship session represents the actual meeting and its outcome.

For example:

Booking

Planned

Mock Interview

November 10, 2026 · 3:00 PM

Mentorship Session

Completed

The mentor has recorded that the interview was conducted, added observations and provided next steps.

Feedback and Review

The student provides feedback on the session and submits a review of the mentor.

This makes your original requirement — the mentor entering the status of an offline session — much clearer.

5. Neo4j: Make the recommendation system a real feature

I particularly recommend investing time here. This is where your project can become more interesting than a typical college mentorship portal.

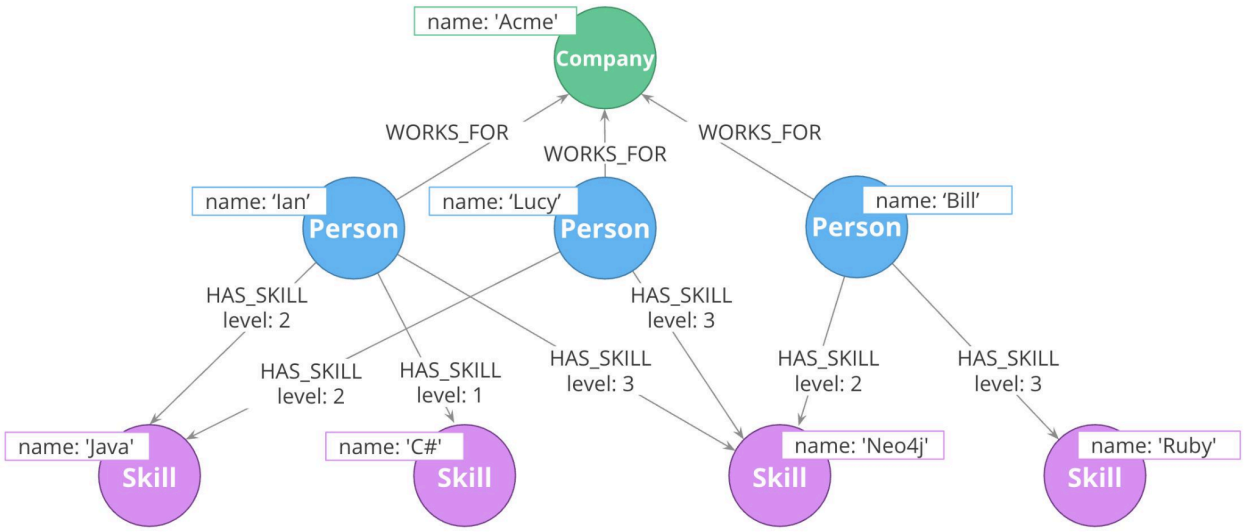
Your syllabus explicitly covers Neo4j nodes, labels, relationships, properties and Cypher. Your matching system is a natural real-world application of those concepts.

5.1 What should the graph represent?

Consider these entities:

- Student
- Mentor
- Skill
- Domain
- Company
- Department

A conceptual graph could look like this:



For your actual model, the relationships should have explicit meanings.

Starting node	Relationship	Ending node
Student	HAS_SKILL	Skill
Mentor	HAS_SKILL	Skill

Student	INTERESTED_IN	Domain
Mentor	SPECIALIZES_IN	Domain
Mentor	WORKS_AT	Company
Student	STUDIES_IN	Department
Mentor	ALUMNI_OF	Department
Student	MENTORED_BY	Mentor

Some relationships should carry their own properties.

For example:

- `HAS_SKILL` can carry proficiency and experience information.
- `MENTORED_BY` can carry relationship status and mentoring history.
- `WORKS_AT` can carry job title and employment period.

This is exactly where Neo4j's data model becomes useful: nodes have labels, relationships have types, and both can have properties.

DBMS Unit 4(2).pdf

5.2 Example: how mentor recommendations should work

Suppose we have the following sample information.

Student profile

Student A

Interested in:

- Machine Learning
- MLOps
- Cloud Computing

Current skills:

- Python
- Machine Learning
- SQL

Available mentors

Tejas N - Software Engineer | Stackforce

Mentor A

ML Engineer · 5 years' experience

Skills: Python, ML, MLOps, Docker

Domain: Machine Learning

Resup - Craft ATS-Ready Resumes by Chatting with AI

Mentor B

Cloud Engineer · 4 years' experience

Skills: AWS, Docker, Kubernetes

Domain: Cloud Computing

Thirupathi Kadari Portfolio

Mentor C

Data Scientist · 3 years' experience

Skills: Python, SQL, Data Science

Domain: Data Analytics

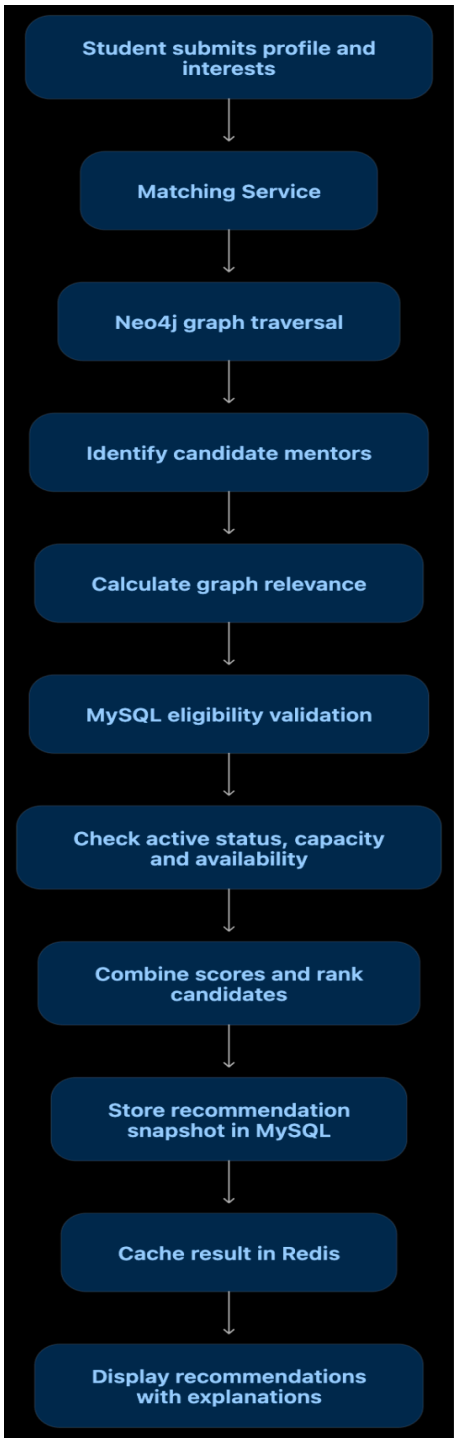
The recommendation engine should not simply search for one matching skill.

It should consider multiple dimensions:

Matching dimension	What it checks
Skill similarity	How many relevant skills overlap?
Domain relevance	Does the mentor specialise in the student's preferred domain?
Experience	How relevant is the mentor's industry experience?
Career relevance	Does the mentor's professional background match the student's goal?
Mentor quality	What are the mentor's historical ratings and outcomes?
Capacity	Is the mentor eligible to accept another student?

These factors should produce an explainable recommendation score. The precise weights should be configurable, rather than permanently hard-coded.

5.3 Recommendation pipeline



The distinction between Neo4j and SQL is important:

Neo4j identifies relevant relationships. MySQL determines whether the recommendation is actually eligible.

For example, Neo4j might discover a mentor who is highly relevant but has already reached their mentorship capacity. MySQL should exclude that mentor from the final results.

5.4 Recommendation explainability

This is a feature I strongly recommend adding.

Instead of displaying:

> Recommended Mentor: Mentor A — 92%

Display:

> Mentor A is recommended because:
>
> - Shares three relevant skills with the student.
> - Specialises in the student's preferred domain.
> - Has five years of relevant industry experience.
> - Has previously mentored students in similar career areas.

This makes the recommendation engine understandable and gives you a good demonstration of graph traversal.

5.5 Keep Neo4j synchronised with MySQL

This is another important engineering decision.

Suppose Mentor A updates their skills.

The flow should be:

1. The profile update is committed to MySQL.
2. The same SQL transaction records an event in a transactional outbox.
3. A background worker reads that event.
4. The worker updates the corresponding Neo4j nodes and relationships.
5. Relevant Redis cache entries are invalidated.
6. The recommendation engine uses the updated graph data.

This approach avoids independently writing to MySQL and Neo4j and hoping both writes succeed.

Neo4j may briefly contain an older representation, so the graph should be treated as an eventually consistent recommendation projection. Booking and allocation decisions must still be validated against MySQL.

6. Redis: Plan actual application-level use cases

I don't want Redis to be included just to demonstrate `SET` and `GET`.

It should solve real application performance problems.

Here is the proposed usage.

Feature	Redis data structure	Purpose
Mentor profile cache	Hash	Store frequently requested mentor information
Domain and skill cache	Hash / Set	Quickly retrieve reference information
Recommendation cache	String / Hash	Cache recent recommendation results
Rate limiting	String / counter	Control repeated API requests
Temporary login challenge	String	Store short-lived verification data
Popular mentor rankings	Sorted Set	Retrieve ranked mentor lists

Recent activity	List or Stream	Maintain a temporary activity feed

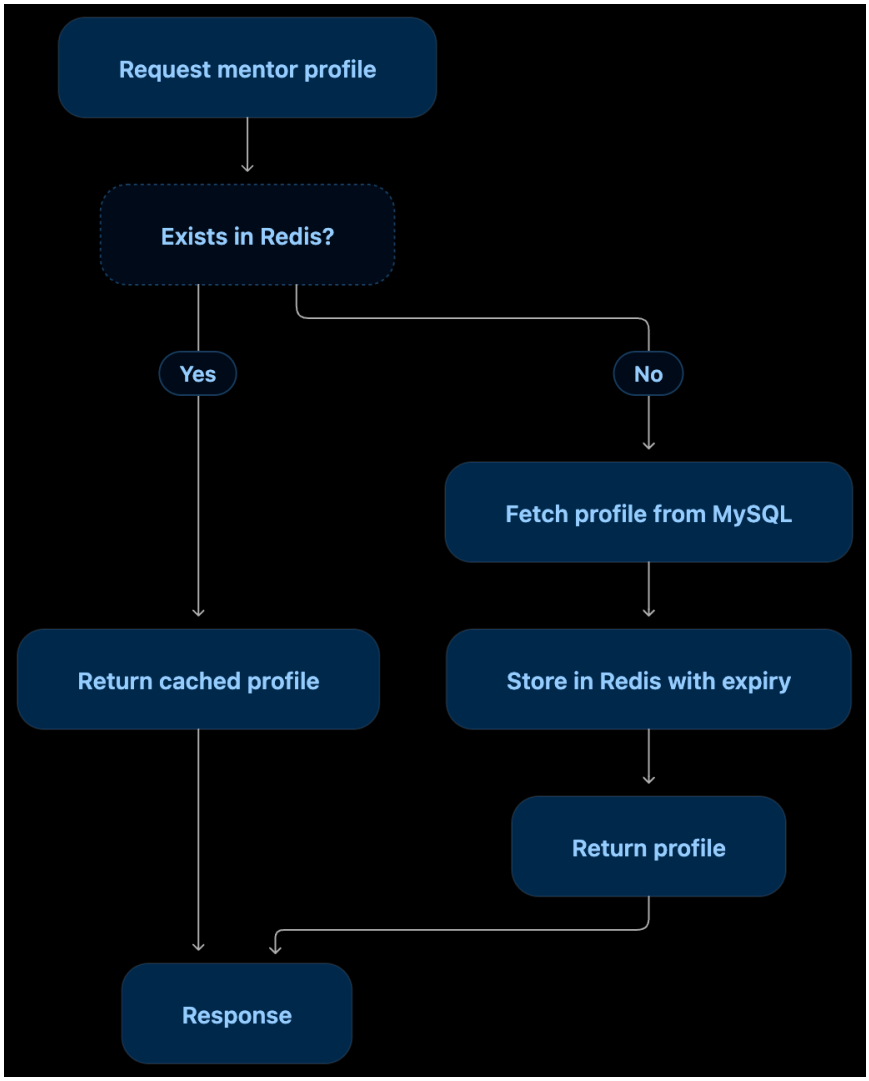
Your DBMS slides cover strings, lists, hashes, sets and sorted sets, alongside TTL and key lifecycle management. These are directly applicable to the proposed design.

DBMS Unit 4(2).pdf

Cache invalidation strategy

For example, suppose a student requests Mentor A's profile.

The application checks Redis first.



When Mentor A updates their professional profile, the relevant cache entries must be invalidated.

I would establish these rules:

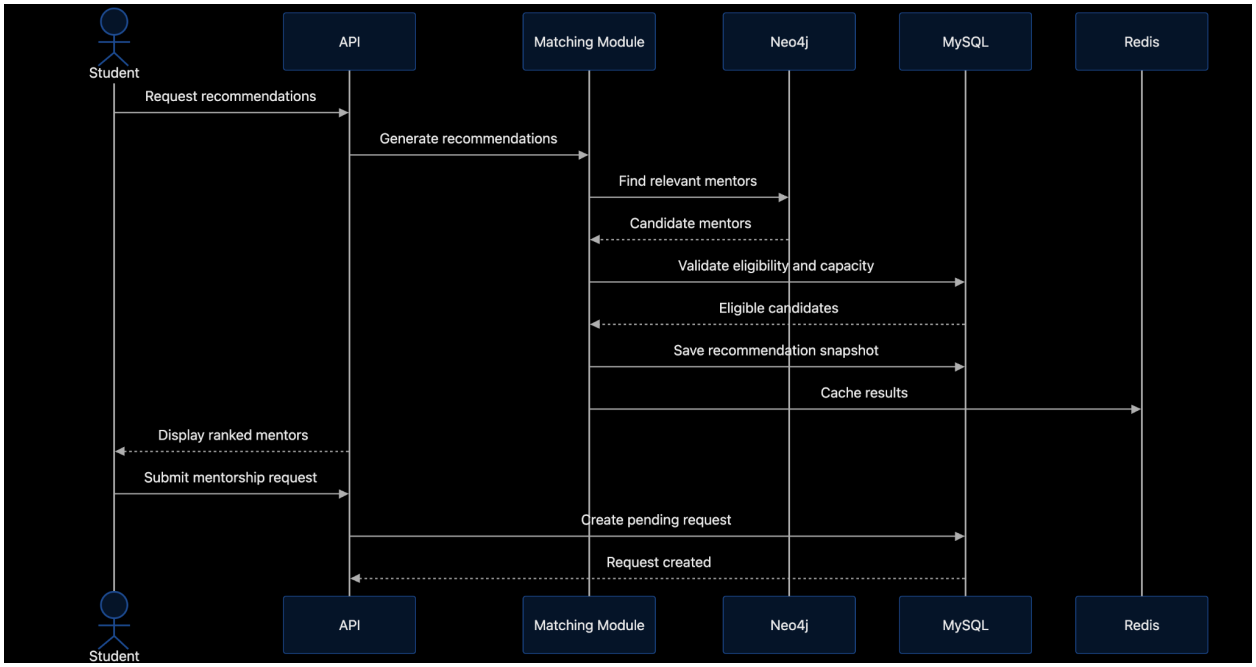
- Use cache-aside for relatively stable profile information.
- Use TTL to avoid indefinitely stale data.
- Invalidate recommendation results when relevant skills or profiles change.
- Do not trust cached booking availability.
- Never let a cache hit bypass MySQL authorization or booking validation.

This will let you demonstrate cache hits, cache misses, TTL, eviction and performance differences.

7. Important business workflows

Now we should connect the different modules into complete application workflows.

Workflow 1: Student discovers and requests a mentor



The request should progress through controlled states:

`Pending → Accepted / Rejected / Cancelled`

An accepted request can produce an allocation.

A rejected or cancelled request should remain in the history for reporting rather than simply disappearing.

Workflow 2: Alumni accepts a request

When a mentor accepts a student request:

- Verify that the request is still pending.
- Verify that the mentor can accept the allocation.
- Prevent duplicate active allocations.
- Create the mentorship allocation.
- Update the request's status.
- Record the event for notifications.

All related changes should occur inside one SQL transaction.

This is a good place to demonstrate stored procedures, transactional integrity and constraints.

Workflow 3: Booking a mock interview

This should be one of your strongest DBMS demonstrations.

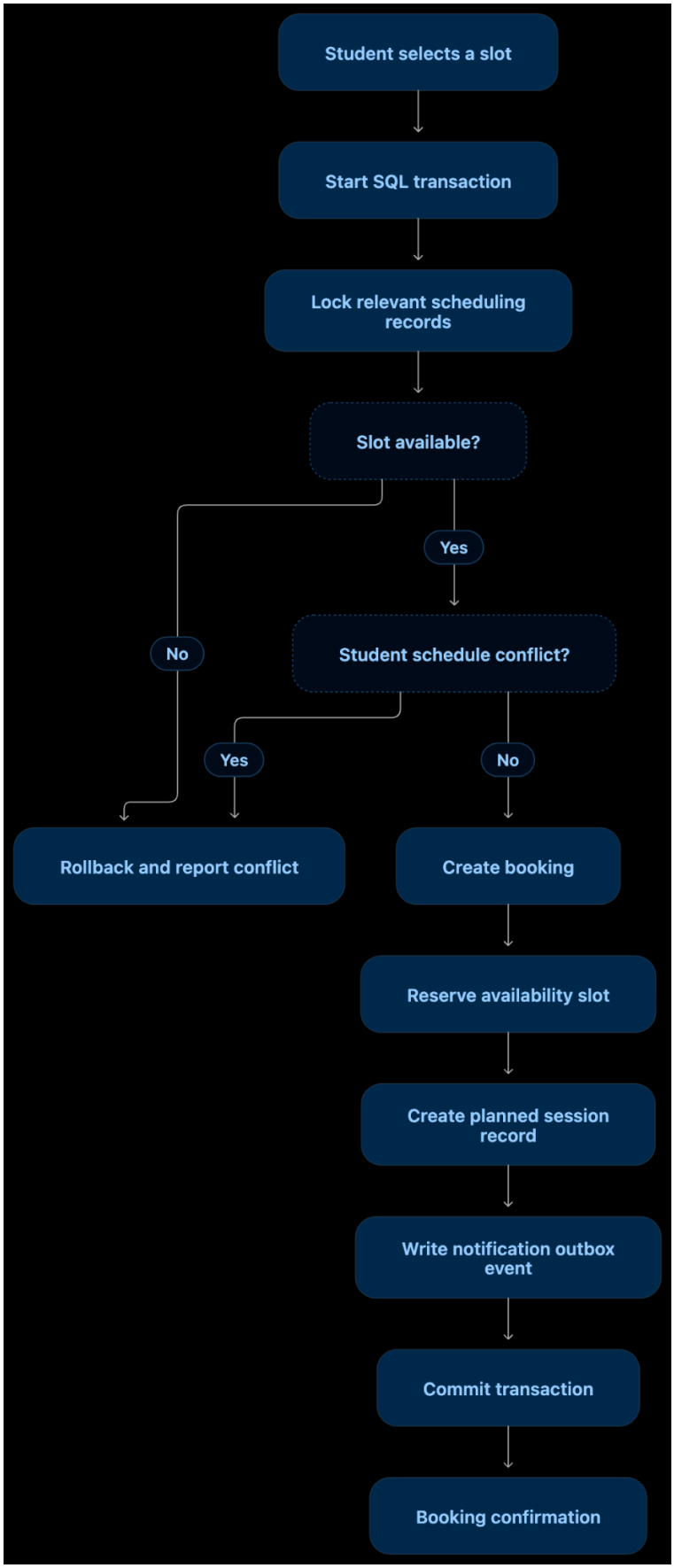
The mentor publishes availability. A student chooses an available slot.

Before confirming the booking, the application checks:

1. The mentor's slot is still available.
2. The student does not already have another overlapping booking.
3. The mentor has not exceeded their capacity.
4. The request or allocation is valid.
5. The booking is not a duplicate submission.

The booking operation must be atomic.

Conceptually:


Student selects a slot
Start SQL transaction
Lock relevant scheduling records
Slot available?
Yes
No
Student schedule conflict?
Yes
No
Rollback and report conflict
Create booking
Reserve availability slot
Create planned session record
Write notification outbox event
Commit transaction
Booking confirmation

Use database uniqueness and transactional locking to prevent concurrent requests from claiming the same slot. Student-side overlapping appointments also need protection, not merely a lock on the mentor's slot.

This connects directly to your Unit 4 topics: transactions, ACID properties, concurrency control and locking. Your slides also discuss problems such as lost updates and the need for controlled concurrent execution.

DBMS Unit 4(2).pdf
DBMS Unit 4(2).pdf

Workflow 4: Offline mock interview and session recording

The application does not need video conferencing integration for its core scope.

After the offline meeting, the mentor records:

- Whether the session was completed.
- The discussion topics.
- Skills assessed.
- Student performance observations.
- Areas for improvement.
- Next steps.
- Recommended preparation before the next session.

Possible session states:

```
Scheduled → Completed / Cancelled / No-show
```

The system should preserve session history.

Workflow 5: Feedback and reviews

I recommend separating these two concepts.

Session feedback

Can contain structured feedback from either participant, including communication, helpfulness, technical preparation and session notes.

Mentor review

Represents the student's rating and review of the mentor after a completed session.

Business rules:

- A session cannot receive feedback before it has taken place.
- Only the actual participants can submit feedback.
- One student can submit at most one mentor review per session.
- A review must reference a valid completed session.
- Review history should not be silently overwritten.

This distinction also makes your reporting more meaningful.

Workflow 6: Notifications

Notifications should be triggered by important business events:

Event	Notification recipient
Mentorship request submitted	Mentor
Request accepted	Student
Request rejected	Student
Booking confirmed	Both participants
Booking cancelled	Both participants
Session reminder	Both participants

Feedback submitted	Relevant participant

Use a transactional outbox pattern: record the notification event in MySQL within the business transaction, then let a background worker deliver it.

This prevents a notification from being sent for an operation that subsequently rolls back.

8. How to demonstrate all four DBMS units

This is where the project becomes particularly valuable academically.

Your syllabus covers introductory relational design, advanced SQL, normalisation and query optimisation, and finally transactions and NoSQL databases. The course also lists labs specifically for SQL, triggers, transaction isolation, Redis CRUD and Neo4j Cypher.

DBMS_U1(3).pdf

DBMS_U1(3).pdf

I would make the project demonstrate every one of those areas.

Unit	Concepts	How AlumniConnect demonstrates them
Unit 1	ER model, relational model, relational algebra, SQL, keys and constraints	Domain modelling, conceptual ER design, primary and foreign keys, integrity constraints
Unit 2	Joins, nested queries, aggregates, views, triggers, procedures, CTEs, windows, full-text search	Mentor discovery, allocation reports, session analytics, profile search
Unit 3	Query processing, optimisation, functional dependencies, normalisation	FD analysis, 1NF–BCNF design, indexing and query-plan comparisons

Unit 4	Transactions, ACID, concurrency, locking, Redis, Neo4j, vector databases	Atomic booking, concurrency testing, caching, graph recommendations and optional semantic search

8.1 Unit 1: Relational design

You should be able to explain:

- Why each entity exists.
- Which relationships are one-to-one, one-to-many and many-to-many.
- Which attributes uniquely identify an entity.
- Why junction entities are needed for student skills and mentor skills.
- How the ER model maps to a relational model.
- Which domain, key, entity-integrity and referential-integrity constraints apply.

8.2 Unit 2: Advanced SQL showcase

Don't write random queries just to increase your SQL query count.

Build queries that solve real platform questions.

SQL concept	Example application question
Joins	Display mentor profiles with their skills, company and domain
Set operations	Combine candidates discovered through different matching methods
NULL handling	Identify alumni with incomplete professional profiles
Aggregates and GROUP BY	Calculate average mentor ratings

HAVING	Find mentors with ratings above a given threshold and sufficient completed sessions
Nested subqueries	Find mentors with experience above the average within their domain
Correlated subqueries	Find students who have not received a mentor allocation
Views	Build a public mentor directory without exposing private information
Triggers	Record important status changes in an audit log
Stored procedures	Process a booking or mentorship acceptance
Functions	Calculate profile completeness or a reusable reporting metric
CTEs	Analyse the request-to-allocation-to-session funnel
Window functions	Rank mentors within each mentorship domain
Full-text search	Search alumni experience descriptions and professional biographies

Your Unit 2 material specifically covers views, correlated nested queries, procedures, CTEs and window functions, so these are particularly good areas to showcase.

DBMS UNIT 2(2).pdf

DBMS UNIT 2(2).pdf

DBMS UNIT 2(2).pdf

DBMS UNIT 2(2).pdf

A particularly useful example is mentor ranking.

Rather than simply returning mentors ordered by average rating, demonstrate rankings within a domain using window functions.

Another good example is a CTE-based analytics report showing:

```
Requests Submitted → Requests Accepted → Allocations Created →
Sessions Completed
```

This is a genuine business question and provides a reason to use advanced SQL.

8.3 Unit 3: Normalisation and query optimisation

I want this unit to be more than an academic explanation in your report.

You should demonstrate the design process.

Normalisation

For the main relational model:

- Identify functional dependencies.
- Identify candidate keys.
- Identify partial dependencies.
- Identify transitive dependencies.
- Demonstrate the relevant decomposition decisions.
- Establish the final normal-form design.

Aim for a well-justified 3NF/BCNF design where applicable. Do not blindly decompose every relation without considering its functional dependencies.

Query optimisation

Select a few meaningful queries, such as:

- Finding all available mentors in a domain.
- Retrieving student mentorship history.
- Ranking alumni based on matching skills.
- Retrieving mentors with the highest feedback ratings.

For each, demonstrate:

1. Initial execution plan.
2. Relevant indexes.
3. Optimised execution plan.
4. Execution-time comparison.
5. Explanation of the difference.

Use `EXPLAIN` and, where available, `EXPLAIN ANALYZE`.

This gives you concrete evidence of query processing and optimisation.

8.4 Unit 4: Advanced database functionality

This unit should have three major demonstrations.

Transaction and concurrency demonstration

Two students attempt to book the same mentor slot simultaneously. Only one booking must succeed.

Redis demonstration

Show that frequently accessed mentor profiles and recommendation results are returned from Redis, and demonstrate cache misses, hits and expiry.

Neo4j demonstration

Show that a student can discover an alumni mentor through shared skills, domains, industry and mentoring relationships.

Vector search extension

Show how a natural-language student goal retrieves relevant alumni profiles even when the wording is different.

This gives you a complete practical connection between the DBMS syllabus and a functioning application.

9. Improvements to your existing mini-project

I would not throw away your previous database work. The existing project should become Version 1 of the redesigned system.

However, I would make these changes.

Existing design	Proposed improvement	Reason
Separate Student and Alumni identity data	Common User identity with separate role profiles	Avoid duplicated authentication information
Student Skill and Alumni Skill	Retain both relationships	Properly represent many-to-many relationships
Mentorship Request	Retain	Core business entity
Mentor Allocation	Retain	Represents an accepted mentorship relationship
No explicit recommendation history	Add Recommendation Run and Candidate Recommendation concepts	Track ranking results and explainability
No dedicated availability concept	Add Mentor Availability	Properly represent available times
Booking and session responsibilities overlap	Separate Booking and Mentorship Session	Distinguish planned meetings from actual outcomes

Feedback contains rating information	Separate Session Feedback and Mentor Review	Different business responsibilities
No notification delivery tracking	Add Notification and Outbox	Reliable notification processing
No dedicated audit history	Add Audit Log	Track important changes

Your previous project already includes meaningful generated test data. The populated-data screenshots show approximately 100 students, 50 alumni, 120 mentorship requests, 90 allocations and 270 mentorship sessions. This is useful for initial functional testing.

DBMS MINIPROJECT.pdf

Later, create larger synthetic datasets specifically for query optimisation and performance demonstrations.

One additional recommendation: recommendation history

This is particularly useful for your project.

Suppose Student A receives five mentor recommendations today.

Next week, the student updates their skills and receives a different set of recommendations.

You should be able to answer:

- What did the platform recommend initially?
- What were the scores?
- Why was a particular mentor recommended?
- Which recommendation did the student select?
- Did that mentorship eventually result in completed sessions?

A recommendation history makes this possible and turns your recommendation engine into something that can be evaluated rather than simply displayed.

10. Scope and project priorities

I would divide the product into two levels so that you do not end up implementing too many advanced features before the main application works.

Level 1 — Core application

Essential

- Student, alumni and administrator portals.
- Authentication and role-based authorization.
- Student and mentor profiles.
- Skills and mentorship-domain management.
- Neo4j-based mentor recommendations.
- Mentorship requests and allocation.
- Availability and booking.
- Offline session tracking.
- Feedback and ratings.
- Notifications.
- SQL analytics and reporting.
- Redis caching.